

The Synthesis Prototype, With Its Supply Chain:

Derivation Replacing Authorship in a Bounded, Receipted Pipeline —

and the Fifty Years of Science Each Layer Stands On

semi-naive saturation → constraint-discovered sequencing → content-addressed execution →
refinement admission

Sean Chatman

with the Praxis working system, `crates/praxis-synthesis` v26.7.2

2 July 2026 — Genesis, Day 9 (revised: the dependency edition)

Abstract

The first edition of this paper described a working prototype, `praxis-synthesis`, in which a five-step plan that Genesis Day 2 authored by hand in PDDL is instead *derived*: from declared capabilities and a saturated fact base, a bounded solver discovers order and parameter bindings, executes the result as a content-addressed DAG, and admits the artifact through refinements that recompute rather than trust. Each research influence got one sentence. This revision pays the debt in full. Every layer of the prototype sits on a lineage of systems and theorems — Datalog’s forty-year arc from Codd through semi-naive evaluation, worst-case-optimal joins, Soufflé, RDFox, QLever, and Nemo; the SAT/SMT arc from Davis–Putnam through CDCL, MiniSat, Z3, difference logic, and minimal-unsatisfiable-core extraction; the content-addressing arc from Merkle trees through Git, the Nix thesis, Bazel, and BLAKE3’s descent from ChaCha; and the refinement-type arc from Freeman–Pfenning through liquid types to Flux — plus the concrete crates the prototype actually links: `blake3`, `serde`, and the in-house `pddl-index`, `chatman-common`, and `prolog8` whose designs are themselves positions taken in those lineages. We keep the ledger discipline of the first edition (theorem / measurement / refusal) and add a fourth label: *lineage* — who discovered it, in what system it shipped, and exactly which slice of it the 8-bounded doctrine takes. The thesis is unchanged and sharpened: **when capabilities are declared rather than plans authored, correctness moves from the author’s head into the derivation chain — and every link of that chain was forged by a community whose receipts are its papers.**

Contents

1	Introduction: a Prototype Is a Position in Several Literatures	2
2	Layer 1 — Datalog: Forty Years from Codd to Nemo	2
2.1	The science	2
2.2	What the prototype takes	3
2.3	Incrementality: the ledger the literature does not ship	4
3	Layer 2 — Constraints: from Davis–Putnam to Certificates	4
3.1	The science	4
3.2	What the prototype takes — and the refusal that defines it	5
4	Layer 3 — Content Addressing: Merkle to Git to Nix to BLAKE3	5
4.1	The science	5
4.2	What the prototype takes	6

5	Layer 4 — Refinements: Freeman–Pfenning to Flux	6
6	The In-House Dependencies, Honestly Described	7
7	Results (Unchanged, Restated with Their Lineage Labels)	7
8	The Refusal Register, with Intellectual Context	8
9	Conclusion: Standing on the Supply Chain, Receipting It	8

1 Introduction: a Prototype Is a Position in Several Literatures

`praxis-synthesis` is 2,500 lines of bounded Rust that compose four capabilities: forward saturation of stratified Horn rules; discovery of capability sequences under eight constraint kinds, with certified refusals; content-addressed, memoized DAG execution; and refinement-style admission. The first edition derived the mathematics from first principles and measured the artifact. What it under-served is the fact that *none of the four ideas is ours*. Each is the load-bearing conclusion of a research lineage, and the prototype’s real design content is the set of choices about *which slice* of each lineage survives contact with the project’s standing doctrine: pure Rust, zero native dependencies, every loop capped, every cap violation a receipted refusal, everything replayable.

The dependency manifest is short, and that is itself a claim:

dependency	kind	carries
<code>pddl-index</code> (in-house)	path	dictionary interning, sorted-ID relations
<code>chatman-common</code> (in-house)	path	BLAKE3 content addressing, rolling chains
<code>blake3</code> (transitively)	crates.io	the hash function itself
<code>serde</code> , <code>serde_json</code>	crates.io	canonical serialization of receipts
<code>thiserror</code>	crates.io	the refusal enum’s error plumbing
<i>refused</i> : <code>z3</code> , <code>cvc5</code>	native	(Section 3)
<i>refused</i> : <code>datafrog/ascent/crepe</code>	crates.io	(Section 2)

Sections 2–5 each proceed the same way: the science and its history; the systems that industrialized it; the concrete dependency or in-house implementation the prototype uses; and what the 8-bounded doctrine kept, dropped, and why. Section 7 restates the measured results; Section 8 re-receipts the refusals, now with their intellectual context.

2 Layer 1 — Datalog: Forty Years from Codd to Nemo

2.1 The science

Lineage 2.1 (Relational foundations, 1970–1977). Datalog’s substrate is Codd’s relational model [1]: data as relations, queries as algebra. The fixpoint extension — queries defined recursively as the least solution of rule sets — entered through logic programming: Kowalski’s predicate-logic-as-programming-language [2] and van Emden–Kowalski’s semantics giving Horn programs a least Herbrand model computed by the immediate-consequence operator T_P [3]. The mathematical warrant is Knaster–Tarski [4]: T_P is monotone on the powerset lattice, so its least fixpoint exists and is the limit of the Kleene chain. Everything Layer 1

proves (Theorem 2.5 of the first edition, restated below) is a finite-universe corollary of a 1955 fixpoint theorem.

Lineage 2.2 (Datalog proper and semi-naive evaluation, 1980s). “Datalog” names the function-symbol-free fragment studied intensively in the 1980s database-theory community; the canonical treatment is Abiteboul–Hull–Vianu [5]. The decade’s two durable algorithmic results are both in the prototype. *Semi-naive evaluation* (Bancilhon; Balbin–Ramamohanarao [6, 7]) observes that a new derivation at round $i+1$ must use at least one premise from round i , so each rule need only be re-joined against the *delta* — work proportional to novelty, not to knowledge. *Stratified negation* (Apt–Blair–Walker [8]) gives negation-as-failure a coherent semantics by layering predicates so that negation only consults strata already saturated; programs that cannot be layered (the win/lose pathology) are rejected. The prototype implements both exactly, and its **Unstratifiable** refusal is the ABW rejection surfaced as a receipt.

Lineage 2.3 (Worst-case-optimal joins, 2012–2018). Pairwise join plans are provably sub-optimal on cyclic queries: for the triangle query $Q(x, y, z) \leftarrow e(x, y), e(y, z), e(x, z)$, any pair-at-a-time plan can materialize $\Theta(N^2)$ intermediate tuples against an output bounded by $O(N^{3/2})$. The bound is Atserias–Grohe–Marx (AGM) [9]; the algorithms meeting it are Ngo–Porat–Ré–Rudra’s NPRR [10] and Veldhuizen’s Leapfrog Triejoin [11], deployed commercially in LogicBlox. The essential move: eliminate *variables*, not *atoms* — bind one variable at a time across all atoms that mention it, intersecting sorted candidate sets. The prototype’s join (Section 2.2) is a bounded cousin: greedy most-bound-first atom ordering with sorted prefix-range probes, which on the triangle pattern reduces the third atom to a single membership probe — the same “never materialize the blowup” effect at ≤ 8 atoms/variables, without the full variable-elimination machinery.

Lineage 2.4 (The engines: Soufflé, R^DFox, Q^Lever, Nemo, and the Rust crates). The 2010s–2020s industrialized Datalog along four axes the prototype consciously samples. *Soufflé* [12] (Oracle Labs lineage; static-analysis workloads, notably Datalog-based points-to analysis) compiles rules to parallel C++ over specialized B-tree/trie relations — the lesson that *representation dominates*. *R^DFox* [13] (Oxford; Motik et al.) demonstrated dense-ID, in-memory, parallel materialization with incremental maintenance. *Q^Lever* [14] (Freiburg; Bast et al.) showed that dictionary-encoding strings to dense integer IDs plus sorted permutations makes knowledge-graph-scale query answering a laptop matter — this is the direct ancestor of praxis’s in-house **pddl-index** crate (“the qlever treatment”), whose **Dict/SymId** interner Layer 1 reuses verbatim. *Nemo* [15] (Dresden; Ivliev et al., ICLP 2023) is the proximate inspiration surfaced by the research sweep: a Rust rule engine handling 10^5 – 10^8 facts on commodity hardware, fully declarative, Apache/MIT — proof that saturation at knowledge-graph scale needs no cluster. On crates.io, **datafrog** (the minimalist semi-naive core once inside Rust’s Polonius borrow checker), **ascent**, and **crepe** (macro-embedded Datalog) were considered and *refused* as dependencies: praxis needs runtime-loaded rules (not compile-time macros), receipted bounds, and its own interned ID space — the intersection was small enough that a 500-line in-crate engine cost less than an integration seam (refusal receipted in Section 8).

2.2 What the prototype takes

From this lineage Layer 1 (**datalog.rs** + **rel.rs**) keeps: T_P and Knaster–Tarski (Theorem 2.5); semi-naive deltas (Proposition 2.6); ABW stratification with the cycle refusal; Q^Lever-style dictionary encoding via **pddl-index::Dict**; Soufflé’s representation lesson as columnar lexicographically sorted **Vec<u32;8>** relations with amortized tail merges; and the WCOJ *intuition* as greedy bound-prefix join ordering. The doctrine adds what none of the

engines ship: **the fixpoint as a value** (fixpoint.hash = BLAKE3 over the sorted structural tuples) and **the closure as a ledger** (Section 2.3) — successive fixpoint hashes chained by fold, so two agents can agree they reasoned from the same closure without exchanging it. One deliberate structural choice removes a whole failure class: with arity capped at 8, an atom’s identity is its *tuple*, ‘(pred, [u32;8])’, not a packed 64-bit hash — the birthday problem that haunts key-packed stores (and that the first edition had to caveat) is eliminated rather than bounded.

Theorem 2.5 (Existence and finiteness; restated). *For a stratified program over a finite interned universe, per-stratum T_P is monotone on $(2^{\text{Atoms}}, \subseteq)$; its least fixpoint exists (Knaster–Tarski) and the ascending chain stabilizes finitely.*

Proposition 2.6 (Semi-naive soundness/completeness; restated). *Semi-naive evaluation with per-position delta restriction derives exactly the naive fixpoint. (Bancilhon’s argument; duplicates are idempotent.)*

2.3 Incrementality: the ledger the literature does not ship

Incremental view maintenance is a mature literature — counting and DRed (delete-rederive) [16], and differential dataflow’s general solution [17] (McSherry et al.; the Rust ecosystem’s deepest entry here). The prototype’s **assert_facts** takes the *additive* half: new facts resume semi-naive rounds from exactly the arrived delta, provably equal to batch re-saturation (test-pinned by identical fixpoint hashes). The *subtractive* half (retraction, and additions under negation — which are retractions in disguise) is where DRed’s complexity lives, and the prototype **refuses** it: asserting into a program with negative rules returns a nonmonotonicity refusal naming the pathology, rather than a wrong closure. What it adds atop the literature is small and, to our knowledge, novel in packaging: each increment’s fixpoint hash is folded onto the last — an audit chain over *reasoned states*, so the history of what was known is as replayable as the history of what was done.

3 Layer 2 — Constraints: from Davis–Putnam to Certificates

3.1 The science

Lineage 3.1 (SAT: DP to CDCL to MiniSat, 1960–2003). Propositional satisfiability’s modern arc runs from the Davis–Putnam procedure [18] and its backtracking refinement DPLL [19] through the conflict-driven clause learning (CDCL) revolution — GRASP (Marques-Silva–Sakallah [20]) introduced learning from conflicts, Chaff (Moskewicz et al. [21]) made unit propagation cheap with watched literals, and MiniSat (Eén–Sörensson [22]) distilled the whole design into a few hundred lines that became the field’s reference implementation. Two CDCL ideas matter to the prototype even though it contains no SAT solver: *propagation before search* (deduce everything cheap before branching) and *learning from failure* (a conflict is an artifact worth keeping). Solver8’s mask propagation is the first idea; its core cache is the second, lifted from clauses to whole problems.

Lineage 3.2 (SMT, DPLL(T), and Z3, 2006–present). Satisfiability modulo theories couples a CDCL core with theory solvers — the DPLL(T) architecture (Nieuwenhuis–Oliveras–Tinelli [23]). Microsoft Research’s Z3 (de Moura–Bjørner [24], 2008) became the field’s workhorse: linear arithmetic, arrays, bitvectors, quantifiers, used everywhere from program verification to the capability-sequencing research [25] that motivated this layer (Köcher et al.: declared capability models compiled to SMT, the solver discovering execution orders and parameter bindings — exactly the prototype’s Layer-2 contract). The theory slice the prototype actually

needs is far smaller than Z3: *difference logic*, constraints of the form $x - y \leq c$, decidable by shortest-path / negative-cycle detection (Bellman–Ford) — the fragment SMT solvers themselves dispatch to graph algorithms [26]. Solver8’s `Before/NotLater/NotEarlier` windows are difference logic over a 16-point timeline, which is why 16-bit masks suffice: the theory is small enough to be a bitwise operation.

Lineage 3.3 (Constraint propagation: AC-3, 1977). Independently of SAT, the constraint-satisfaction community built propagation as its primitive: Mackworth’s AC-3 [27] repeatedly revises variable domains against constraints until fixpoint, pruning before (or instead of) search. Solver8’s window propagation *is* AC-3 with domains as `u16` bitmasks and revision as mask intersection/shift — the doctrine’s contribution is only the observation that horizon ≤ 16 makes every domain a machine word, so a full propagation round is a handful of ALU operations per constraint.

Lineage 3.4 (Unsat cores and MUS extraction, 2000s–present). The product Layer 2 actually ships — refusals as certificates — has its own literature. Modern SAT solvers emit unsatisfiability proofs (DRAT [28]) that independent checkers verify; the celebrated 200TB Pythagorean-triples proof [29] made “machine-checkable impossibility” a headline result. A *minimal unsatisfiable subset* (MUS) is an unsat core no proper subset of which is unsat; the standard deletion-based extractor (try removing each constraint; keep it iff satisfiability appears) is folklore formalized across [30, 31] with a rich engineering literature on doing better than n solver calls. The 8-bounded doctrine makes the naive extractor *optimal enough*: with ≤ 64 constraints and propagation-only unsat checks costing microseconds, deletion-based MUS is ≤ 64 mask fixpoints. The prototype’s tests pin true minimality — every proper subset of an emitted core fails to certify — which is the MUS definition verified extensionally.

3.2 What the prototype takes — and the refusal that defines it

Solver8 keeps: AC-3 as bitmask propagation; difference-logic windows; deletion-based MUS with named culprits; a mandatory-set analysis (sole-producer closure under `Requires`) that turns *missing facts* into certificates too — the self-application demo refuses to schedule `push` with core `[MissingFact(authorization)]`, no search performed. The defining refusal: `no z3, no cvc5`. Both are mature native libraries; linking them breaks pure-Rust reproducible builds and imports megabytes of solver whose theories the problem does not need. The counter-bet, receipted: an 8-bounded kernel whose propagation vectorizes and whose refusals replay is worth more *at fleet scale* than a general solver whose answers are oracular. The `Solver` trait remains the seam; if the falsifier sweep had broken the differential, that receipt would have argued for taking the native dependency after all. It did not break (Section 7).

4 Layer 3 — Content Addressing: Merkle to Git to Nix to BLAKE3

4.1 The science

Lineage 4.1 (Hash trees and authenticated data, 1979–1990s). Merkle’s 1979 thesis and the certified-digital-signature paper [32] introduced the hash tree: name data by hashes of hashes, and membership, equality, and tamper-evidence become $O(\log n)$ comparisons. Every subsequent content-addressed system is a Merkle construction with a use case attached. The prototype’s DAG receipt is two of them at once: the rolling chain ($h^+ = H(h^- \parallel \text{frame})$), a Merkle *list* — the same shape as a blockchain, used here without consensus theater) and the order-independent root (a Merkle *set*: hash of sorted `node:output` pairs).

Lineage 4.2 (Content-addressed software: Git, Nix, Bazel, 2003–2020). Three engineering traditions proved that naming artifacts by their content transforms system behavior. *Git* (Torvalds, 2005; the object model descends from Monotone’s) made every blob, tree, and commit a hash of its content — integrity, deduplication, and distributed merge fall out of the naming scheme [33]. *Nix* (Dolstra’s 2006 thesis [34]) applied the same move to *builds*: a package’s store path is a hash of everything that produced it, making builds reproducible, rollback trivial, and “works on my machine” a checkable proposition — the purely-functional deployment model NixOS and Guix industrialized. *Bazel* (Google’s Blaze lineage [35]) and its remote-execution protocol demonstrated the action cache at monorepo scale: an action keyed by (command, input digests) never runs twice, organization-wide. The prototype’s **MemoCache** key — $\text{ca}(a_v \parallel \bar{h}_v)$, action hash concatenated with sorted input output-hashes — is exactly Bazel’s action key and Nix’s derivation hash, shrunk to a struct. *OxyMake* [36], the research sweep’s find, packages the same discipline as a single-binary Rust workflow engine with BLAKE3 naming and TLA+ specs — proximate confirmation that the pattern wants to live in Rust.

Lineage 4.3 (The hash itself: from ChaCha to BLAKE3, 2008–2020). The prototype’s only cryptographic dependency is the **blake3** crate (via in-house **chatman-common**). Its pedigree: Bernstein’s ChaCha stream cipher [37] contributed the ARX core; BLAKE (Aumasson et al.) built a SHA-3 finalist around it; BLAKE2 [38] optimized it into the fastest widely-trusted hash of the 2010s; and BLAKE3 [39] (O’Connor, Aumasson, Neves, Wilcox-O’Hearn, 2020) restructured it as a Merkle tree internally — unbounded parallelism, incremental verification, one function for hashing/KDF/MAC/XOF. The security contract the prototype leans on is collision resistance at 256 bits: for N objects, collision probability $\leq N^2/2^{257}$; at $N = 10^{12}$ lifetime objects, $< 10^{-53}$ — ignorable against hardware failure. (The first edition’s honest caveat about 64-bit *atom-key* collisions is now moot: Layer 1’s structural identity removed the packed key from the trust path entirely.)

Lineage 4.4 (Serialization as a trust surface: *serde*). Receipts are only as canonical as their bytes. **serde** [40] (Tolnaru/Tolnay lineage; the Rust ecosystem’s serialization framework) provides derive-generated, allocation-conscious, schema-stable JSON via **serde_json**. The prototype’s discipline: every hashed frame is serialized through one code path, so “same value, same bytes, same hash” is a property of the derive, not of programmer care. **thiserror** carries the **Refusal** enum’s error implementation — refusals serialize like any other receipt, which is what lets an unsat core cross the **synth/v1** wire intact.

4.2 What the prototype takes

Layer 3 keeps: Merkle list + Merkle set (chain and root, with distinct jobs: history vs. artifact identity); the Nix/Bazel action key as the memo key; content-based node identity so the root is declaration-order invariant (Theorem 4.2 of the first edition, test-pinned); and BLAKE3 via **chatman_common::provenance** (**content_address**, **fold_event**, **genesis_seed** — domain-separated genesis values, Merkle hygiene the ad-hoc systems of the 2000s learned painfully). The fleet measurement (Section 7) is this lineage’s payoff quantified: with deliberation memoized by content, marginal cost tracks novelty — Nix’s promise (“never build the same thing twice”), restated for *reasoning* and measured.

5 Layer 4 — Refinements: Freeman–Pfenning to Flux

Lineage 5.1 (Refinement and liquid types, 1991–2023). Refinement types — types qualified by predicates, $\{v : \text{int} \mid v \geq 0\}$ — enter with Freeman–Pfenning [42]. Liquid types (Rondon–Kawaguchi–Jhala [43], 2008) made them practical by restricting predicates to conjunctions of

templates so inference reduces to fixpoint computation over an SMT oracle; LiquidHaskell [44] carried the discipline into a production language. Flux [45] (Lehmann–Geller–Vazou–Jhala, PLDI 2023) is the Rust instantiation and the sweep’s fourth find: refinements ride Rust’s ownership system, so aliasing — the classic killer of automated verification — is discharged by the borrow checker, halving annotation burden relative to program-logic tools like Prusti.

Lineage 5.2 (The transposition — and the differential tradition). The prototype does not verify its Rust with Flux (a receipted deferral: Flux is a research tool with its own toolchain; the crate’s invariants are currently enforced by types + tests). It *transposes* the discipline: refinements as machine-checkable predicates over *artifacts* — plan-respects-horizon, DAG-acyclic, chain-recomputes, fixpoint-closed, plan-reaches-goal — carried with the object and re-checked by admission, never short-circuited. The re-checking style has its own lineage worth naming: differential testing (McKeeman [46]) and the fuzzing tradition from Miller’s original crash study [47] through csmith [48] and simdjson’s oracle-pair method [49]. The verifier’s PlanReachesGoal is a second implementation of plan semantics; the falsifier sweep (P6) is fuzzing with the Solver8-vs-oracle differential as fitness — both are McKeeman’s insight operationalized: *two independent computations that must agree are a bug detector with no false negatives on disagreement*.

6 The In-House Dependencies, Honestly Described

Three path dependencies are praxis’s own, and each is itself a position in the lineages above:

pddl-index (v26.7.2). “The clever treatment” for PDDL grounding: Dict string interner ($\rightarrow$ dense u32 SymIds), sorted per-predicate tuple relations, XOR-filter membership pruning (Graf-Lemire [41]), and a delete-relaxed reachability fixpoint that is precisely a Datalog least fixpoint specialized to action grounding. Layer 1 reuses its interner and generalizes its fixpoint; the historical order matters — the grounder came first, the general engine is its child.

chatman-common (v26.7.2). The provenance kernel: `content_address`, `fold_event`, `RollingChain`, domain-separated genesis seeds, and ed25519 signed receipts. It is the project’s single point of Merkle hygiene — every praxis chain, from law receipts to Genesis seals to this crate’s DAG receipts, folds through the same forty lines.

prolog8 (v26.7.1; sibling, *not* a dependency of this crate). The bounded backward-chaining Horn kernel (SLD, arity/body/vars ≤ 8 , depth ≤ 32 , proof-carrying receipts, replay). Layer 1 exists because prolog8 *cannot* close large recursive relations — by design: point queries with proofs, not saturation. The two engines are complementary halves of the same doctrine, and the 100-node transitive-closure measurement is the boundary between them made executable.

7 Results (Unchanged, Restated with Their Lineage Labels)

Measurement 7.1 (Saturation — the Bancilhon/ABW/QLever stack). Transitive closure over a 100-node chain: exactly $\binom{100}{2} = 4,950$ derived paths, multi-round semi-naive convergence, beyond prolog8’s depth-32 SLD by construction. Scale receipt (release, under concurrent load, recorded as observed): triangles differential-verified against an independent hash-join oracle at $10^5/4 \times 10^5/10^6$ edges (1.2s / 12.8s / 24.0s); 64-ary-forest closure $10^5 \rightarrow 295,773$ derived (2.4s), $10^6 \rightarrow 3,729,468$ (126s). Known next lever, receipted: per-round merge re-sort; the columnar layout is already what index joins want.

Measurement 7.2 (Certificates — the GRASP/MUS stack). Conflicting windows over the lawobject domain yield a core of exactly the conflicting triple; both decoys excluded; every proper subset fails `verify_core`; the cached proof replays for later agents (`replayed: true`). The self-application domain refuses `push` with `[MissingFact(authorization)]` pre-search, while the brute oracle needed `nodes_explored > 0` to fail — a proof versus a failure-to-find, live over the `synth/v1` wire.

Measurement 7.3 (The overlap curve — the Nix/Bazel stack, applied to reasoning). $N = 10,000$ pipelines from K templates, shared memo + core caches (release): work/pipeline 892.26 at novelty 1.0; 88.94 at 0.1; 8.61 at 0.01; 0.87 at 0.001; 0.00 at full overlap — throughput $1,898 \rightarrow 39,055$ pipelines/s. Work per pipeline $\approx 892 \times (K/N)$: **a fleet’s reasoning cost grows with its novelty, not its size**. The first measurement run exposed the missing plan cache (cost flat at ~ 890 for all $K > 1$) — the receipt caught its own gap, which is the methodology working.

Measurement 7.4 (The falsifier — the McKeeman/simdjson stack). 256 adversarial cells over 4 generations of discrepancy-guided mutation (8-axis seeded domains: dead-end traps, delete interference, derived-predicate preconditions, constraint load): differential *held* — 218 solved-agree (byte-equal plans and costs, Solver8 never searching more), 37 refused-agree, 1 lawful asymmetry (Solver8 succeeded where brute exhausted budget; plan independently replayed). Optimal plans never entered a trap.

8 The Refusal Register, with Intellectual Context

Refusal 8.1 (No native SMT (z3/cvc5)). Not a rejection of the DPLL(T) lineage — a bet that its difference-logic slice, 8-bounded and bitwise, plus MUS-certified refusals, serves a receipted fleet better than an oracular general solver. The Solver trait is the seam; the falsifier receipt is the standing evidence the bet has not yet lost.

Refusal 8.2 (No Datalog crate (datafrog/ascent/crepe)). All three are honorable members of the lineage; none offers runtime rule loading + receipted caps + praxis’s interned ID space without an adapter thicker than the engine. Soufflé/Nemo-class engines are C++/separate processes. The in-crate engine is ~ 500 lines against a forty-year-settled specification — the rare case where reimplementing is the *low-risk* path, because the theorems are the spec.

Refusal 8.3 (No differential-dataflow; no retraction). Additions-only incrementality with a refusal on nonmonotonic cases, rather than DRed/counting or timely-dataflow generality. Salvage: the ledger chain is agnostic to how the delta was computed; a DRed-style maintainer can slot beneath it later.

Refusal 8.4 (No Flux-the-tool; refinements transposed, not typed). The crate’s invariants are enforced by Rust types and a 44-test suite, not by liquid typing; Flux’s discipline is applied to artifacts instead. Salvage: the refinement predicates are exactly the properties a future Flux annotation pass would state.

Refusal 8.5 (No Nemo-scale claim). Measured to 10^6 EDB facts on contended hardware, not 10^8 . The storage is already the sorted dense-ID form the big engines use; the receipt names the merge-scheduling lever that comes next.

9 Conclusion: Standing on the Supply Chain, Receipting It

The first edition closed: *authorship placed correctness in a head; derivation places it in a chain*. This edition adds the acknowledgment the chain deserves: every link was forged elsewhere. Tarski gave the fixpoint; Bancilhon the delta; Apt, Blair and Walker the strata;

Atserias, Grohe, Marx, Ngo, Porat, Ré and Rudra the join bound; Mackworth the propagation; Marques-Silva the learning-from-conflict; the MUS community the minimal core; Merkle the tree; Torvalds and Dolstra the addressed artifact; Aumasson and colleagues the hash; Freeman, Pfenning, Jhala and the Flux group the refinement discipline; McKeeman the differential. The prototype’s own contribution is deliberately narrow: an 8-bounded selection from each lineage, composed so that every stage emits a content address, every refusal carries a certificate, and the whole run is one hash. Narrow is the point. **The science is the supply chain; the doctrine is the admission gate; the receipts are ours.**

References

- [1] E. F. Codd. *A Relational Model of Data for Large Shared Data Banks*. CACM 13(6), 1970.
- [2] R. Kowalski. *Predicate Logic as Programming Language*. IFIP 1974.
- [3] M. van Emden, R. Kowalski. *The Semantics of Predicate Logic as a Programming Language*. JACM 23(4), 1976.
- [4] A. Tarski. *A Lattice-Theoretical Fixpoint Theorem and Its Applications*. Pacific J. Math. 5, 1955.
- [5] S. Abiteboul, R. Hull, V. Vianu. *Foundations of Databases*. Addison–Wesley, 1995. (Chs. 12–15.)
- [6] F. Bancilhon. *Naive Evaluation of Recursively Defined Relations*. In *On Knowledge Base Management Systems*, 1986.
- [7] I. Balbin, K. Ramamohanarao. *A Generalization of the Differential Approach to Recursive Query Evaluation*. J. Logic Programming 4(3), 1987.
- [8] K. Apt, H. Blair, A. Walker. *Towards a Theory of Declarative Knowledge*. In *Foundations of Deductive Databases and Logic Programming*, 1988.
- [9] A. Atserias, M. Grohe, D. Marx. *Size Bounds and Query Plans for Relational Joins*. FOCS 2008 / SIAM J. Comput. 2013.
- [10] H. Ngo, E. Porat, C. Ré, A. Rudra. *Worst-Case Optimal Join Algorithms*. PODS 2012 / JACM 2018.
- [11] T. Veldhuizen. *Leapfrog Triejoin: A Simple, Worst-Case Optimal Join Algorithm*. ICDT 2014.
- [12] H. Jordan, B. Scholz, P. Subotić. *Soufflé: On Synthesis of Program Analyzers*. CAV 2016.
- [13] B. Motik, Y. Nenov, R. Piro, I. Horrocks, D. Olteanu. *Parallel Materialisation of Datalog Programs in Centralised, Main-Memory RDF Systems*. AAAI 2014.
- [14] H. Bast, B. Buchhold. *QLever: A Query Engine for Efficient SPARQL+Text Search*. CIKM 2017.
- [15] A. Ivliev et al. *Nemo: First Glimpse of a New Rule Engine*. ICLP 2023; arXiv:2308.15897.
- [16] A. Gupta, I. Mumick, V. Subrahmanian. *Maintaining Views Incrementally*. SIGMOD 1993.

- [17] F. McSherry, D. Murray, R. Isaacs, M. Isard. *Differential Dataflow*. CIDR 2013.
- [18] M. Davis, H. Putnam. *A Computing Procedure for Quantification Theory*. JACM 7(3), 1960.
- [19] M. Davis, G. Logemann, D. Loveland. *A Machine Program for Theorem-Proving*. CACM 5(7), 1962.
- [20] J. Marques-Silva, K. Sakallah. *GRASP: A Search Algorithm for Propositional Satisfiability*. IEEE Trans. Computers 48(5), 1999.
- [21] M. Moskewicz, C. Madigan, Y. Zhao, L. Zhang, S. Malik. *Chaff: Engineering an Efficient SAT Solver*. DAC 2001.
- [22] N. Eén, N. Sörensson. *An Extensible SAT-solver*. SAT 2003.
- [23] R. Nieuwenhuis, A. Oliveras, C. Tinelli. *Solving SAT and SAT Modulo Theories: from DPLL to DPLL(T)*. JACM 53(6), 2006.
- [24] L. de Moura, N. Bjørner. *Z3: An Efficient SMT Solver*. TACAS 2008.
- [25] A. Köcher et al. *Automating the Generation of Planning Problems from Semantic Capability Models*. arXiv:2312.08801.
- [26] S. Cotton, O. Maler. *Fast and Flexible Difference Constraint Propagation for DPLL(T)*. SAT 2006.
- [27] A. Mackworth. *Consistency in Networks of Relations*. Artificial Intelligence 8(1), 1977.
- [28] M. Heule, W. Hunt, N. Wetzler. *Trimming While Checking Clausal Proofs*. FMCAD 2013.
- [29] M. Heule, O. Kullmann, V. Marek. *Solving and Verifying the Boolean Pythagorean Triples Problem via Cube-and-Conquer*. SAT 2016.
- [30] A. Belov, J. Marques-Silva. *MUSer2: An Efficient MUS Extractor*. JSAT 8, 2012.
- [31] J. Marques-Silva, C. Mencía. *Reasoning About Inconsistent Formulas*. IJCAI 2020 (survey).
- [32] R. Merkle. *A Certified Digital Signature*. CRYPTO 1989 (thesis work 1979).
- [33] L. Torvalds et al. *Git*. 2005–. Content-addressed object store (blobs/trees/commits by SHA-1, later SHA-256).
- [34] E. Dolstra. *The Purely Functional Software Deployment Model*. PhD thesis, Utrecht University, 2006.
- [35] Google. *Bazel and the Remote Execution API*. 2015–. Content-keyed action cache at monorepo scale.
- [36] *OxyMake: Content-Addressable Workflow Automation in Rust*. arXiv:2606.20989, 2025.
- [37] D. J. Bernstein. *ChaCha, a Variant of Salsa20*. SASC 2008.
- [38] J.-P. Aumasson, S. Neves, Z. Wilcox-O’Hearn, C. Winnerlein. *BLAKE2: Simpler, Smaller, Fast as MD5*. ACNS 2013.

- [39] J. O'Connor, J.-P. Aumasson, S. Neves, Z. Wilcox-O'Hearn. *BLAKE3: One Function, Fast Everywhere*. 2020.
- [40] D. Tolnay et al. *Serde: Serialization Framework for Rust*. 2015–.
- [41] T. M. Graf, D. Lemire. *Xor Filters: Faster and Smaller Than Bloom and Cuckoo Filters*. ACM JEA 25, 2020.
- [42] T. Freeman, F. Pfenning. *Refinement Types for ML*. PLDI 1991.
- [43] P. Rondon, M. Kawaguchi, R. Jhala. *Liquid Types*. PLDI 2008.
- [44] N. Vazou, E. Seidel, R. Jhala, D. Vytiniotis, S. Peyton-Jones. *Refinement Types for Haskell*. ICFP 2014.
- [45] N. Lehmann, A. Geller, N. Vazou, R. Jhala. *Flux: Liquid Types for Rust*. PLDI 2023; arXiv:2207.04034.
- [46] W. McKeeman. *Differential Testing for Software*. Digital Technical Journal 10(1), 1998.
- [47] B. Miller, L. Fredriksen, B. So. *An Empirical Study of the Reliability of UNIX Utilities*. CACM 33(12), 1990.
- [48] X. Yang, Y. Chen, E. Eide, J. Regehr. *Finding and Understanding Bugs in C Compilers*. PLDI 2011.
- [49] G. Langdale, D. Lemire. *Parsing Gigabytes of JSON per Second*. VLDB J. 28, 2019.